

UNIVERSIDAD EL BOSQUE

Ingeniería de Sistemas
Bases de Datos I

DOCUMENTO DE ANÁLISIS DE RDBMS

Angela Maria Reina Vivas
Angel Felipe Alvarez Benavides
Cristian David Vargas Cetina

Asignatura: Bases de Datos I
Docente: Christian Felipe Duarte Arevalo

Bogotá D.C.
2026

1. INTRODUCCIÓN

El presente documento describe el análisis realizado para la selección del Sistema Gestor de Bases de Datos Relacional (RDBMS) utilizado en el proyecto Gym, desarrollado como un sistema de gestión integral para gimnasio.

El sistema tiene como objetivo administrar la información relacionada con clientes, instructores, planes de membresía, suscripciones, sesiones de clase, asistencias, facturación y pagos, permitiendo centralizar y organizar los datos del gimnasio de manera eficiente.

Para la selección del motor de base de datos se evaluaron diferentes alternativas de uso común en el desarrollo de aplicaciones empresariales, entre ellas PostgreSQL, MySQL y Microsoft SQL Server. El análisis tuvo en cuenta aspectos como compatibilidad con Spring Boot y JPA/Hibernate, facilidad de integración, soporte de transacciones, rendimiento, licenciamiento y herramientas disponibles para el desarrollo.

Como resultado del análisis realizado, PostgreSQL fue seleccionado como el RDBMS más adecuado para las necesidades del proyecto, debido a su estabilidad, robustez, compatibilidad con el ecosistema Java y disponibilidad como software libre.

2. CONTEXTO DEL SISTEMA

2.1 Descripción General del Proyecto

Gym es un sistema de gestión desarrollado para administrar la información de un gimnasio. El proyecto fue implementado utilizando Spring Boot y PostgreSQL, permitiendo gestionar clientes, instructores, planes de membresía, suscripciones, clases, asistencias, facturación y pagos.

El sistema utiliza una arquitectura basada en capas, separando la lógica de presentación, la lógica de negocio y la persistencia de datos. Para la conexión entre la aplicación y la base de datos se utilizó Spring Data JPA junto con Hibernate.

Dentro del sistema existen diferentes relaciones entre las entidades principales:

- Un cliente puede tener múltiples suscripciones a lo largo del tiempo.
- Cada suscripción está asociada a un único plan.
- Una suscripción puede generar facturas relacionadas con los pagos del servicio.
- Una factura puede tener un pago asociado.
- Un pago puede registrarse mediante distintos métodos como efectivo, tarjeta o transferencia.
- Las sesiones de clase tienen un instructor asignado y un cupo definido.

- Los clientes pueden registrar asistencia a las sesiones programadas.

2.2 Perfil de Carga del Sistema

Los volúmenes estimados del sistema fueron definidos tomando como referencia el comportamiento operativo promedio de gimnasios pequeños y medianos reportado en plataformas especializadas del sector fitness. Según estudios y artículos de gestión de gimnasios, un gimnasio puede administrar desde decenas hasta miles de socios dependiendo de su tamaño, servicios ofrecidos y ubicación geográfica.

Con base en estas referencias y considerando el alcance académico del proyecto Gym, se estimaron los siguientes volúmenes de datos y patrones de acceso:

Entidad	Volumen Estimado (1 año)	Patrón de Acceso
Cliente	500 – 2.000 registros	Lectura frecuente, escritura moderada
Suscripción	500 – 5.000 registros	Escritura al contratar, lectura constante
Factura / Pago	500 – 5.000 registros	Escritura frecuente, consultas analíticas
Sesión	1.000 – 10.000 registros/año	Lectura muy frecuente
Asistencia	5.000 – 50.000 registros/año	Alta escritura, consultas de ranking
Pago Mixto (Efectivo/Tarjeta/Transferencia)	Variable por método	Reportes agregados frecuentes

Estas estimaciones permiten dimensionar adecuadamente la necesidad de un sistema gestor de bases de datos robusto, con soporte para integridad transaccional, consultas analíticas y crecimiento progresivo de información.

2.3 Stack Tecnológico

El sistema Gym fue desarrollado utilizando un stack tecnológico moderno basado en Java y Spring Boot, orientado a aplicaciones empresariales con arquitectura multicapa y persistencia relacional. La selección de tecnologías se realizó priorizando compatibilidad, estabilidad, facilidad de integración y soporte con PostgreSQL.

El stack tecnológico implementado en el proyecto es el siguiente:

Componente	Tecnología Utilizada
Backend	Java 17 + Spring Boot 3.3.x
Persistencia / ORM	Spring Data JPA + Hibernate 6.x
Frontend	HTML5 + CSS3 + JavaScript ES6 Modules

Componente	Tecnología Utilizada
Gestión automática del esquema	Hibernate ddl-auto=update
Dialecto ORM	org.hibernate.dialect.PostgreSQLDialect
Conector JDBC	org.postgresql.Driver (PostgreSQL JDBC Driver 42.x)
Servidor de aplicaciones	Spring Boot Embedded Tomcat
Sistema operativo de desarrollo	Windows 11 / Linux

La integración entre Spring Boot, Hibernate y PostgreSQL representa una de las combinaciones más utilizadas en aplicaciones empresariales Java debido a su alto nivel de compatibilidad y estabilidad. Hibernate proporciona soporte nativo para PostgreSQL mediante el dialecto PostgreSQLDialect, permitiendo generar consultas SQL optimizadas y aprovechar funcionalidades específicas del motor como tipos de datos avanzados, soporte transaccional ACID y operaciones eficientes sobre claves autogenerated.

Adicionalmente, el uso de Spring Data JPA reduce significativamente la complejidad de acceso a datos mediante repositorios automáticos y consultas derivadas, mientras que el driver JDBC oficial de PostgreSQL garantiza conectividad estable y soporte actualizado para Java 17 y Spring Boot 3.x.

3. CANDIDATOS EVALUADOS

Para la selección del sistema gestor de bases de datos relacional del proyecto GymDB se analizaron tres de las soluciones más utilizadas en entornos empresariales y académicos: PostgreSQL, MySQL y Microsoft SQL Server. La evaluación consideró aspectos relacionados con rendimiento, compatibilidad con el stack tecnológico del proyecto, soporte transaccional, licenciamiento, facilidad de integración y capacidades avanzadas del motor.

3.1 PostgreSQL 16

PostgreSQL es un sistema de gestión de bases de datos objeto-relacional de código abierto con más de 35 años de desarrollo activo. Es mantenido por la comunidad internacional PostgreSQL Global Development Group y distribuido bajo la PostgreSQL License, una licencia permisiva similar a BSD que permite su utilización libre en proyectos académicos y comerciales.

PostgreSQL es reconocido por su cumplimiento de estándares SQL, su robusto soporte transaccional ACID y sus capacidades avanzadas de integridad y concurrencia mediante MVCC (Multi-Version Concurrency Control). Además, ofrece funcionalidades avanzadas como:

- Consultas complejas y optimización avanzada.
- Soporte para tipos de datos personalizados y JSON.
- Índices avanzados y búsquedas eficientes.
- Funciones y procedimientos almacenados.
- Alta compatibilidad con Hibernate y Spring Data JPA.

De acuerdo con el ranking DB-Engines, PostgreSQL se encuentra entre los motores de bases de datos más utilizados y con mayor crecimiento sostenido a nivel mundial. Asimismo, en encuestas de desarrolladores como Stack Overflow Developer Survey, PostgreSQL ha sido catalogado como uno de los motores más valorados por la comunidad de desarrollo profesional.

Estas características lo convierten en una opción especialmente adecuada para sistemas empresariales construidos con Java y Spring Boot.

3.2 MySQL 8.0

MySQL es uno de los sistemas gestores de bases de datos relacionales más populares del mundo y es propiedad de Oracle Corporation desde 2010. Se distribuye bajo licencia GPL v2, aunque también cuenta con versiones comerciales.

Su popularidad se debe principalmente a su facilidad de uso, amplio soporte comunitario y adopción masiva en aplicaciones web tradicionales, especialmente dentro del ecosistema LAMP (Linux, Apache, MySQL y PHP).

Entre sus principales características se encuentran:

- Facilidad de instalación y administración.
- Buen rendimiento en aplicaciones web de lectura intensiva.
- Amplia documentación y soporte comunitario.
- Compatibilidad con herramientas y frameworks populares.

Sin embargo, aunque MySQL ofrece soporte transaccional y compatibilidad con JPA/Hibernate, algunas funcionalidades avanzadas de PostgreSQL relacionadas con consultas complejas, manejo avanzado de tipos de datos y extensibilidad son más limitadas en comparación.

3.3 Microsoft SQL Server 2022

Microsoft SQL Server es un sistema gestor de bases de datos empresarial desarrollado por Microsoft y orientado principalmente a entornos corporativos basados en tecnologías Microsoft y .NET.

SQL Server ofrece una arquitectura robusta, herramientas avanzadas de administración y una fuerte integración con servicios empresariales de Microsoft, incluyendo Azure y Power BI. Entre sus características principales destacan:

- Alto rendimiento en entornos empresariales.
- Herramientas avanzadas de análisis y administración.
- Integración con ecosistemas Microsoft.
- Soporte transaccional y alta disponibilidad.

No obstante, SQL Server posee un modelo de licenciamiento comercial en sus versiones más completas, lo cual puede representar costos importantes para proyectos pequeños o académicos. Además, aunque existe soporte para Linux y Java, su integración suele ser más natural dentro de entornos .NET y Windows Server.

4. CRITERIOS DE EVALUACIÓN

La selección del sistema gestor de bases de datos para GymDB se realizó mediante una evaluación comparativa basada en siete criterios técnicos y funcionales. Cada criterio recibió un peso porcentual de acuerdo con su importancia dentro de los requerimientos del proyecto y del stack tecnológico utilizado.

#	Criterio	Peso	Justificación
1	Compatibilidad con Spring Boot / JPA / Hibernate	25%	El proyecto fue desarrollado completamente sobre el ecosistema Java y Spring Boot, por lo que la integración con Hibernate y Spring Data JPA representa un factor crítico.
2	Soporte de integridad referencial y transacciones ACID	20%	El sistema administra procesos financieros como pagos, facturación y suscripciones, por lo que se requiere consistencia transaccional y confiabilidad en los datos.
3	Licenciamiento y costo	15%	El proyecto se desarrolla en un contexto académico sin presupuesto destinado a licencias comerciales o infraestructura propietaria.
4	Rendimiento en consultas analíticas / JOINS complejos	15%	El sistema genera reportes y consultas agregadas sobre múltiples tablas relacionadas, especialmente en módulos financieros y de asistencia.
5	Soporte de tipos de datos avanzados	10%	Se requiere soporte adecuado para enumeraciones, manejo de fechas, estructuras complejas y extensibilidad futura del modelo de datos.
6	Comunidad, documentación y madurez	10%	La disponibilidad de documentación técnica, soporte comunitario y estabilidad del motor son importantes para mantenimiento y aprendizaje continuo.
7	Portabilidad y despliegue en entornos cloud	5%	Aunque el proyecto se desarrolla localmente, se considera importante la facilidad de despliegue futuro

			en entornos cloud y contenedores.
--	--	--	-----------------------------------

La ponderación asignada prioriza principalmente la integración con el ecosistema Spring Boot y la capacidad del motor para garantizar integridad y consistencia transaccional, debido a que estos aspectos representan los requisitos más críticos del sistema GymDB.

5. ANÁLISIS COMPARATIVO DETALLADO

5.1 Compatibilidad con Spring Boot, JPA e Hibernate

Este criterio posee el mayor peso dentro de la evaluación (25%), debido a que el backend del sistema GymDB fue desarrollado utilizando Spring Boot 3.3.x, Spring Data JPA y Hibernate 6.x como proveedor ORM principal. La compatibilidad requerida no se limita únicamente a la conexión JDBC, sino también al soporte del dialecto ORM, manejo de tipos de datos Java, generación automática de identificadores y compatibilidad con consultas nativas utilizadas en el proyecto.

Aspecto	PostgreSQL	MySQL	SQL Server
Dialecto Hibernate oficial	PostgreSQLDialect	MySQLDialect	SQLServerDialect
Soporte SERIAL / IDENTITY	SERIAL	AUTO_INCREMENT	IDENTITY
Compatibilidad con @Enumerated(EnumType.STRING)	Completa	Limitada en algunos escenarios	Completa
Operador ILIKE (búsqueda case-insensitive)	Nativo	No disponible	No disponible
Soporte de CAST y funciones SQL avanzadas	Completo	Completo	Completo
Compatibilidad con @Query(nativeQuery=true)	Completa	Completa	Completa
Soporte java.time (LocalDate, LocalTime, LocalDateTime)	Nativo	Requiere configuración adicional	Nativo
Driver JDBC oficial	postgresql-42.x	mysql-connector-java	mssql-jdbc

El proyecto Gym implementa consultas nativas específicas de PostgreSQL, particularmente el operador ILIKE para búsquedas case-insensitive en el módulo de instructores:

```
SELECT *
FROM instructor
WHERE primer_nombre ILIKE CONCAT('%', :nombre, '%');
```

El operador ILIKE es una característica nativa de PostgreSQL que permite búsquedas sin distinción entre mayúsculas y minúsculas de forma directa y eficiente. En motores como

MySQL y SQL Server habría sido necesario reemplazar esta funcionalidad utilizando expresiones como:

LOWER(primer_nombre) LIKE LOWER(:nombre)

Esto introduce mayor complejidad en las consultas y puede afectar el aprovechamiento de índices en algunos escenarios.

Adicionalmente, PostgreSQL presenta una integración especialmente estable con Hibernate 6.x mediante el dialecto PostgreSQL dialect, permitiendo utilizar de forma transparente características como:

- Mapeo automático de tipos java.time.
- Generación de IDs mediante SERIAL e IDENTITY.
- Soporte eficiente para transacciones y consultas complejas.
- Compatibilidad avanzada con consultas JPQL y SQL nativo.

Debido a estas ventajas, PostgreSQL obtuvo la mejor valoración dentro del criterio de compatibilidad con Spring Boot y tecnologías ORM.

5.2 Integridad Referencial y Soporte ACID

El sistema GymDB administra operaciones financieras y transaccionales críticas, incluyendo pagos mixtos, facturación, suscripciones y actualización de estados de pago. Debido a esto, el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) representa un requisito fundamental para garantizar la integridad y confiabilidad de la información almacenada.

Propiedad ACID	PostgreSQL	MySQL (InnoDB)	SQL Server
Atomicidad (@Transactional)	Completa	Completa con InnoDB	Completa
Consistencia mediante FOREIGN KEY	Siempre activo	Dependiente del motor y configuración	Estricta
Niveles de aislamiento	Completo	Completo	Completo
Durabilidad de transacciones	WAL nativo y robusto	Redo log	Transaction Log
Soporte MVCC	Nativo	Disponible en InnoDB	Disponible
Aplicación de CHECK constraints	Estricta	Limitada en versiones antiguas	Estricta

En el proyecto GymDB, el servicio PagoService implementa el método registrarPagoMixto(), el cual se encuentra anotado con @Transactional y ejecuta múltiples operaciones dependientes dentro de una misma transacción:

- Validación de existencia de factura.
- Verificación de estado de pago.

- Registro del pago principal.
- Registro de métodos de pago asociados.
- Actualización del estado de la factura.

```
@Transactional
public void registrarPagoMixto(PagoDTO pagoDTO) {
    // operaciones de pago
}
```

PostgreSQL garantiza que todas estas operaciones se ejecuten como una única unidad atómica. Si ocurre un error durante cualquier paso del proceso, el motor realiza automáticamente un rollback completo de la transacción, evitando inconsistencias parciales en la información financiera.

Adicionalmente, PostgreSQL implementa de forma nativa el mecanismo MVCC (Multi-Version Concurrency Control), el cual permite manejar múltiples transacciones concurrentes sin bloquear innecesariamente las operaciones de lectura, mejorando tanto la consistencia como el rendimiento del sistema.

Aunque MySQL ofrece soporte transaccional mediante el motor InnoDB, históricamente presentó limitaciones relacionadas con la aplicación estricta de restricciones CHECK y diferencias de comportamiento según el motor de almacenamiento utilizado. PostgreSQL, en cambio, mantiene un comportamiento consistente y estricto en la validación de restricciones e integridad referencial.

Debido a la importancia de la consistencia financiera y la integridad transaccional dentro del sistema Gym, PostgreSQL obtuvo una valoración superior en este criterio de evaluación.

5.3 Licenciamiento y Costo

El proyecto Gym fue desarrollado en un contexto académico dentro de la Universidad El Bosque, por lo que el costo de licenciamiento y las restricciones de uso representan un criterio importante tanto para el entorno de desarrollo como para una posible implementación futura en producción.

Aspecto de Costo	PostgreSQL	MySQL	SQL Server
Licencia	Gratuito	GPL v2 (libre) / Licencia comercial si se combina con software propietario.	Licencia comercial. Express gratuita con límites (10 GB, sin agente SQL, no HA).
Costo producción (pequeño negocio)	Sin costo de licencia	\$0 GPL / Variable comercial	\$3.586 USD/año (Standard) o más
Restricciones de uso	Ninguna.	GPL puede requerir abrir código fuente en uso comercial.	Express limitada a 10 GB. HA solo en versiones de pago.
Soporte en la nube (SaaS)	AWS RDS, GCP Cloud SQL, Azure Database, Supabase, Neon, Railway	AWS RDS, GCP Cloud SQL, Azure Database, PlanetScale	Azure SQL Database, AWS RDS SQL Server (mayor costo)

PostgreSQL se distribuye bajo la PostgreSQL License, una licencia permisiva similar a BSD que permite:

- Uso libre en proyectos académicos y comerciales.
- Modificación y distribución sin restricciones.
- Integración con software propietario sin obligación de liberar código fuente.

En contraste, MySQL utiliza licencia GPL v2 para su versión comunitaria. Aunque esto permite su uso gratuito, pueden surgir restricciones legales en ciertos escenarios comerciales o de integración con software propietario, especialmente cuando se requiere distribución cerrada o soporte empresarial.

Por otro lado, Microsoft SQL Server posee un modelo de licenciamiento principalmente comercial. Aunque existe una edición SQL Server Express gratuita, esta presenta limitaciones importantes para entornos reales de producción, incluyendo:

- Tamaño máximo de base de datos.
- Restricciones de rendimiento.
- Ausencia de funcionalidades avanzadas de alta disponibilidad.
- Limitaciones en herramientas administrativas empresariales.

Adicionalmente, PostgreSQL cuenta con amplio soporte en plataformas cloud modernas como:

- AWS RDS.
- Google Cloud SQL.
- Azure Database for PostgreSQL.
- Supabase.
- Neon.
- Railway.

Esto facilita una futura migración del proyecto hacia infraestructura cloud sin costos de licenciamiento adicionales.

Debido a su modelo completamente libre, ausencia de restricciones legales y alta flexibilidad de despliegue, PostgreSQL obtuvo la mejor valoración en el criterio de licenciamiento y costo.

5.4 Rendimiento en Consultas Analíticas y JOINS Complejos

El módulo de Reportes de GymDB realiza consultas analíticas complejas que involucran múltiples JOINS entre tablas (cliente → suscripción → factura → pago → métodos de pago), funciones de agregación (SUM, COUNT, AVG, MAX), agrupaciones (GROUP BY) y subconsultas UNION ALL. El rendimiento en este tipo de consultas es fundamental.

Ejemplos reales del sistema que demandan alto rendimiento analítico:

- rankingClientesPagos: JOIN de 4 tablas (pago → factura → suscripcion → cliente) con GROUP BY y ORDER BY.
- distribucionMetodosPago: UNION ALL de 3 tablas (efectivo, tarjeta, transferencia) con COUNT.
- ingresosPorPlan: JOIN de 3 tablas (plan → suscripcion → factura) con SUM y filtro de estado.

- sesionesMasAsistidas: JOIN de sesion y asistencia con COUNT y ORDER BY descendente.

PostgreSQL destaca en este criterio por varias razones técnicas:

- Planificador de consultas (Query Planner): PostgreSQL posee uno de los planificadores de consultas más sofisticados entre los RDBMS open source. Utiliza estadísticas de distribución de datos y genera planes de ejecución adaptados al volumen y cardinalidad de los datos, eligiendo entre Hash Join, Merge Join o Nested Loop según el caso.
- Índices avanzados: Soporta B-Tree, Hash, GiST, GIN y BRIN. Los índices parciales y funcionales permiten optimizar consultas específicas como búsquedas de facturas PAGADAS sin necesidad de recorrer toda la tabla.
- Parallel Query: PostgreSQL 16 soporta ejecución paralela de consultas para operaciones de aggregation en tablas grandes, distribuyendo la carga en múltiples workers.
- MVCC (Multi-Version Concurrency Control): Permite lecturas concurrentes sin bloqueos de escritura, esencial cuando múltiples usuarios acceden simultáneamente al sistema (registro de asistencias mientras se generan reportes).

Estudios comparativos publicados por Percona y la comunidad PostgreSQL (2022-2023) muestran que PostgreSQL supera a MySQL entre un 15% y un 40% en consultas analíticas complejas con múltiples JOINS y agregaciones, mientras que MySQL mantiene ventaja en operaciones simples de lectura/escritura de punto (OLTP puro). Dado que GymDB tiene un perfil mixto OLTP/OLAP (transacciones y reportes), PostgreSQL es la opción más equilibrada.

5.5 Soporte de Tipos de Datos Avanzados

El sistema utiliza tipos de datos específicos que requieren soporte nativo en el motor de base de datos:

Tipo de Dato	PostgreSQL	MySQL	SQL Server
ENUM personalizado (EstadoAsistencia, EstadoFactura, FranquiciaTarjeta, TipoTarjeta)	Tipo propio + VARCHAR almacenado como String con @Enumerated(EnumType.STRING)	ENUM nativo pero poco flexible (no se puede alterar sin recrear)	Solo VARCHAR/NVARCHAR. Sin ENUM nativo.
LocalDate / LocalTime / LocalDateTime (java.time)	DATE, TIME, TIMESTAMP con timezone completo	TIME no maneja microsegundos. Problemas con timezone en versiones anteriores.	DATE, TIME, DATETIME2 con alta precisión

DECIMAL(10,2) para montos financieros	NUMERIC(10,2) con precisión exacta	DECIMAL(10,2) nativo	DECIMAL(10,2) nativo
TEXT / VARCHAR ilimitado para beneficios y descripciones	TEXT sin límite real	TEXT limitado; LONGTEXT necesario para >64KB	NVARCHAR(MAX) sin límite
SERIAL / BIGSERIAL para IDs autogenerados	SERIAL, BIGSERIAL, SEQUENCES	AUTO_INCREMENT	IDENTITY(1,1)
JSON / JSONB para datos semiestructurados	JSON y JSONB (indexable) nativo	JSON nativo (no indexable de forma nativa)	JSON desde SQL Server 2016

El manejo de los tipos java.time (LocalDate, LocalTime, LocalDateTime) en el sistema es extensivo. Hibernate 6.x + PostgreSQL gestiona nativamente la conversión entre estos tipos Java y los tipos DATE, TIME y TIMESTAMP de PostgreSQL sin necesidad de conversores adicionales. En MySQL, especialmente en versiones anteriores a la 8.0, existen problemas conocidos con el manejo de timezone en tipos DATETIME que han causado bugs difíciles de diagnosticar en aplicaciones Spring Boot.

5.6 Comunidad, Documentación y Madurez

La madurez de un sistema gestor de bases de datos se refleja en la estabilidad de su código base, la calidad de su documentación oficial, la frecuencia de actualizaciones y el tamaño de su comunidad técnica. Estos factores son importantes porque influyen directamente en la facilidad de aprendizaje, la disponibilidad de soporte técnico y la sostenibilidad del sistema a largo plazo.

Indicador	PostgreSQL	MySQL	SQL Server
Años en el mercado	1989	1995	1989
Repositorio GitHub (Stars aprox. 2024)	14.5K+	10K+ (fork MySQL)	N/A (código cerrado)
Preguntas en Stack Overflow	~120.000 preguntas (postgresql tag)	~650.000 (mysql tag)	~165.000 (sql-server tag)
Stack Overflow Survey 2023 — Más usado por profesionales	45.5% (1er lugar)	40.7% (2do lugar)	25.8% (3er lugar)
Stack Overflow Survey 2023 — Más deseado	49.1% (1er lugar)	16.3%	12.0%

Ciclo de release principal	Anual (major) + menor frecuentes	Anual (major) + menor frecuentes	Anual (major)
Soporte LTS	5 años por versión mayor	5 años por versión mayor	7-10 años por versión

PostgreSQL es reconocido en la industria por la calidad y profundidad de su documentación oficial, disponible en el portal oficial de PostgreSQL. La documentación cubre desde conceptos básicos de SQL hasta temas avanzados como optimización de consultas, replicación, concurrencia, índices especializados, extensiones y desarrollo de funciones almacenadas.

Además, PostgreSQL cuenta con una comunidad técnica muy activa que contribuye constantemente con mejoras, correcciones de seguridad y nuevas funcionalidades. La publicación de nuevas versiones mayores ocurre de manera anual, acompañada de actualizaciones menores frecuentes enfocadas en estabilidad y seguridad.

En el ecosistema Java y Spring Boot, PostgreSQL posee una integración altamente madura y ampliamente documentada, lo que facilita el uso de Hibernate, Spring Data JPA y JDBC sin configuraciones complejas. Esto representa una ventaja importante para proyectos académicos y empresariales, ya que existe una gran cantidad de tutoriales, foros, ejemplos y documentación técnica disponible.

Aunque MySQL también posee una comunidad extremadamente grande debido a su popularidad histórica en aplicaciones web, PostgreSQL ha mostrado un crecimiento sostenido en adopción profesional durante los últimos años, especialmente en sistemas empresariales, aplicaciones analíticas y arquitecturas modernas basadas en microservicios y cloud computing.

5.7 Portabilidad y Despliegue en Entornos Cloud

Aunque el proyecto está desplegado localmente (localhost:5432), es importante considerar la facilidad de migración a entornos cloud o de contenedorización para escalabilidad futura.

- PostgreSQL está disponible como imagen oficial en Docker Hub (postgres:latest), lo que facilita el despliegue en contenedores con docker-compose o Kubernetes.
- Los principales proveedores cloud ofrecen servicios gestionados de PostgreSQL: Amazon RDS for PostgreSQL, Google Cloud SQL for PostgreSQL, Azure Database for PostgreSQL, Supabase (PostgreSQL como servicio con APIs REST automáticas) y Neon (PostgreSQL serverless).
- Herramientas de migración de esquema como Flyway y Liquibase tienen soporte de primera clase para PostgreSQL, facilitando la gestión del ciclo de vida del esquema en producción.
- La variable de entorno DATABASE_URL en formato postgresql://user:pass@host:port/db es el estándar en plataformas PaaS como Heroku, Railway y Render.

6. MATRIZ COMPARATIVA FINAL

La siguiente tabla presenta la evaluación ponderada de cada candidato en los 7 criterios definidos, con calificación de 1 a 5 estrellas y puntaje final calculado:

Criterio (Peso)	PostgreSQL	Puntaje	MySQL	Puntaje	SQL Server	Puntaje
Compatibilidad Spring/JPA/Hibernate (25%)	5/5	1.25	4/5	1.00	4/5	1.00
Integridad / ACID (20%)	5/5	1.00	4/5	0.80	5/5	1.00
Licenciamiento / Costo (15%)	5/5	0.75	4/5	0.60	2/5	0.30
Rendimiento analítico / JOINS (15%)	5/5	0.75	3/5	0.45	4/5	0.60
Tipos de datos avanzados (10%)	5/5	0.50	3/5	0.30	4/5	0.40
Comunidad / Documentación (10%)	5/5	0.50	4/5	0.40	4/5	0.40
Portabilidad / Cloud (5%)	5/5	0.25	5/5	0.25	4/5	0.20
PUNTAJE FINAL PONDERADO (100%)		5.00 / 5.00		3.80 / 5.00		3.90 / 5.00

7. RAZONES ESPECÍFICAS DE SELECCIÓN PARA GYMDB

Más allá del análisis comparativo general entre PostgreSQL, MySQL y SQL Server, existen razones directamente relacionadas con la implementación técnica de GymDB que justifican de manera concreta la selección de PostgreSQL como sistema gestor de bases de datos del proyecto.

7.1 Uso del Operador ILIKE

El repositorio de instructores implementa búsquedas de texto case-insensitive mediante el siguiente query nativo:

```
SELECT*
FROM                                instructor
WHERE primer_nombre ILIKE CONCAT('%', :nombre, '%');
```

El operador ILIKE es una característica específica de PostgreSQL que permite realizar comparaciones de texto ignorando diferencias entre mayúsculas y minúsculas. Esta funcionalidad simplifica las consultas y mantiene una sintaxis clara y legible.

En motores como MySQL o SQL Server sería necesario utilizar expresiones adicionales como:

LOWER(primer_nombre) LIKE LOWER(:nombre)

Este enfoque incrementa la complejidad de las consultas y puede afectar el aprovechamiento de índices tradicionales en determinadas configuraciones.

7.2 Dialecto Explícito en application.properties

La configuración del proyecto define explícitamente el dialecto de PostgreSQL dentro de Hibernate:

```
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

El uso de este dialecto permite que Hibernate genere consultas SQL optimizadas específicamente para PostgreSQL, incluyendo:

- Manejo eficiente de claves autogenerated.
- Compatibilidad completa con SERIAL y IDENTITY.
- Uso correcto de tipos DATE, TIME y TIMESTAMP.
- Compatibilidad con consultas nativas y funciones específicas del motor.
- Generación automática de DDL compatible con PostgreSQL.

Esto reduce problemas de compatibilidad y evita configuraciones adicionales dentro de Spring Boot.

7.3 Uso de COALESCE en Queries de Reportes

El módulo de reportes utiliza funciones de agregación sobre tablas que pueden no contener registros en determinados escenarios. Para evitar valores NULL, se utiliza la función SQL COALESCE:

```
SELECT      COALESCE(SUM(e.monto),0)
FROM efectivo e;
```

PostgreSQL implementa COALESCE de manera completamente compatible con el estándar SQL y optimiza este tipo de operaciones analíticas de forma eficiente.

Esto resulta especialmente importante en reportes financieros, donde el sistema debe retornar valores numéricos válidos incluso cuando no existen pagos registrados para un método específico.

7.4 Uso de CAST Explícito para ENUMs

El sistema almacena múltiples enumeraciones Java mediante @Enumerated(EnumType.STRING), por ejemplo:

- EstadoFactura
- EstadoAsistencia

- TipoTarjeta
- FranquiciaTarjeta

En algunas consultas nativas se utiliza conversión explícita de tipos:

```
WHERE f.estado = CAST(:estado AS varchar)
```

Este comportamiento es totalmente compatible con PostgreSQL y con la forma en que Hibernate 6 administra parámetros tipados en consultas SQL nativas.

La compatibilidad entre PostgreSQL y Hibernate reduce errores de conversión de tipos durante la ejecución de consultas complejas.

7.5 Compatibilidad con GenerationType.IDENTITY y SERIAL

Todas las entidades principales del sistema utilizan generación automática de identificadores:

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

PostgreSQL soporta esta estrategia de forma nativa mediante:

- SERIAL
- BIGSERIAL
- GENERATED ALWAYS AS IDENTITY

Esto permite una integración directa con Hibernate sin necesidad de secuencias manuales ni configuraciones adicionales.

La estrategia IDENTITY es especialmente útil en aplicaciones CRUD como GymDB, donde múltiples entidades requieren claves primarias autogeneradas de forma segura y concurrente.

8. REFERENCIAS Y FUENTES

Las afirmaciones técnicas y comparativas presentadas en este documento se basan en documentación oficial y estudios reconocidos de la industria:

1. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
2. Stack Overflow. (2023). *Stack Overflow Developer Survey 2023 — Databases*. <https://survey.stackoverflow.co/2023/#most-popular-technologies-database>
3. DB-Engines. (2024). *DB-Engines Ranking*. <https://db-engines.com/en/ranking>
4. Percona. (2023). *Open Source Data Management Software Survey*. <https://www.percona.com/open-source-data-management-software-survey>
5. Oracle Corporation. (2024). *MySQL 8.0 Reference Manual — CHECK Constraints*. <https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html>

6. Microsoft. (2024). *SQL Server 2022 Licensing Guide*. <https://www.microsoft.com/en-us/sql-server/sql-server-2022-pricing>
 7. Hibernate ORM. (2024). *Hibernate 6.x User Guide — Dialects*. [https://docs.jboss.org/hibernate/orm/6.6/userguide/html_single/Hibernate User Guide.html](https://docs.jboss.org/hibernate/orm/6.6/userguide/html_single/Hibernate%20User%20Guide.html)
 8. Spring. (2024). *Spring Data JPA Reference Documentation*. <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
 9. Momjian, B. (2001). *PostgreSQL: Introduction and Concepts*. Addison-Wesley.
 10. Riggs, S., & Ciolli, G. (2023). *PostgreSQL 16 Administration Cookbook*. Packt Publishing.
-

9. CONCLUSIÓN

Tras el análisis técnico y comparativo realizado, PostgreSQL se consolida como la alternativa más adecuada para el proyecto GymDB. El motor ofrece compatibilidad completa con el stack tecnológico utilizado en el sistema, compuesto por Java 17, Spring Boot 3.x, Spring Data JPA e Hibernate 6.x.

Las características de PostgreSQL permiten cubrir de manera eficiente los requerimientos funcionales y no funcionales del sistema, especialmente en aspectos relacionados con integridad transaccional, consultas analíticas, soporte de tipos avanzados y rendimiento en operaciones con múltiples relaciones.

Asimismo, la implementación actual del proyecto ya se encuentra estrechamente alineada con PostgreSQL mediante:

- El uso del dialecto PostgreSQLDialect.
- El driver JDBC oficial org.postgresql.Driver.
- Queries nativas específicas como ILIKE.
- Estrategias de generación de IDs compatibles con SERIAL.
- Soporte completo para transacciones mediante @Transactional.

Además de sus capacidades técnicas, PostgreSQL representa una solución sostenible desde el punto de vista económico y académico, gracias a su licencia completamente libre, amplia documentación y fuerte respaldo de la comunidad global de desarrolladores.

Por estas razones, PostgreSQL no solo satisface los requerimientos actuales de GymDB, sino que también proporciona una base sólida para futuras ampliaciones del sistema y una posible migración hacia entornos de producción empresariales o cloud.